

Rafael Parreira Chequer

rafaelpchequer@gmail.com

Documento de Visão para o Sistema Stream Sentry

14 de Novembro de 2025

Proposta do aluno Rafael Parreira Chequer ao curso de Engenharia de Software como projeto de Trabalho de Conclusão de Curso (TCC), tendo como orientadores os professores: Cleiton Silva Tavares, Danilo de Quadros Maia Filho, Leonardo Vilela Cardoso e Raphael Ramos Dias Costa.

OBJETIVOS

Este documento apresenta a visão do sistema **Stream Sentry**, uma plataforma web para automação de testes end-to-end e monitoramento de qualidade em aplicações de videoconferência baseadas em WebRTC. O sistema é composto por um frontend em React (TypeScript), um servidor de orquestração em Go, um runner de automação em Node.js/Puppeteer e persistência em MongoDB. Ele visa atender às demandas de desenvolvedores, engenheiros de Quality Assurance (QA) e equipes ágeis, promovendo maior eficiência no ciclo de desenvolvimento de software por meio da automação de simulações de carga com múltiplos usuários virtuais, da coleta de telemetria WebRTC em tempo real e da integração com diferentes provedores de vídeo (Jitsi, Whereby e Zoom, além de salas WebRTC genéricas).

ESCOPO

O **Stream Sentry** é uma ferramenta web distribuída sob licença MIT que automatiza testes *end-to-end* e validação de *performance* para aplicações de videoconferência. Suas principais competências técnicas incluem:

- **Automação e Simulação Otimizada:** Utiliza *headless browsers* (Puppeteer) otimizados para simular múltiplos usuários virtuais simultâneos rodando em hardware padrão, extinguindo a necessidade de testes manuais exaustivos.

-
- **Integração Multi-API Intercambiável:** Suporta aplicações baseadas em diferentes tecnologias de vídeo através de uma arquitetura baseada no padrão de projeto *Strategy*, permitindo a execução de testes "out-of-the-box" em WebRTC nativo, Whereby, Zoom SDK e Jitsi de forma modular.
 - **Telemetria em Tempo Real:** Coleta métricas de baixo nível (latência, perdas de pacote, *bitrate*) diretamente das instâncias virtuais, transmitindo os dados de forma assíncrona via WebSockets para a atualização instantânea de gráficos e *logs* no *Dashboard*.
 - **Análise Estruturada (DX):** Produz relatórios analíticos detalhados do histórico de testes, exportáveis em formatos estruturados (JSON e CSV), garantindo alta interoperabilidade com ferramentas de análise de terceiros.

O sistema é voltado para equipes técnicas que buscam integrar a automação de testes de multimídia a fluxos ágeis. Ele substituirá processos manuais de validação, que são demorados e propensos a falhas no diagnóstico de rede e mídia.

Sistemas atuais/concorrentes: O cenário atual de ferramentas de teste para aplicações de vídeo apresenta diversas abordagens, cada uma com limitações específicas que o Stream Sentry busca resolver

- **Ferramentas de diagnóstico de baixo nível (ex.: `chrome://webrtc-internals`, `rtc-stats API`):** Fornecem dados brutos essenciais (como *bitrate*, *jitter* e perda de pacotes) gerados pelas conexões *peer-to-peer*. Contudo, operam de forma estritamente manual e individual. O Stream Sentry automatiza a orquestração simultânea de múltiplos usuários virtuais, agregando a extração de dados do `rtc-stats` e centralizando-os em um *Dashboard* em tempo real.
- **Ferramentas genéricas de E2E (ex.: Selenium, Cypress, Playwright):** Projetadas para simular interações *web* tradicionais, não possuem arquitetura nativa para WebRTC. A utilização dessas ferramentas para testes de vídeo exige a elaboração de *scripts* complexos (*workarounds*) para injeção de mídia simulada (*fake media*) e para a extração manual das métricas de latência. O Stream Sentry possui suporte nativo e otimizado para

1. `webrtc-internals` — ferramenta interna do Chrome: <https://webrtc.org/>
2. WebRTC Statistics API (`getStats`): <https://www.w3.org/TR/webrtc-stats/>
3. Selenium: <https://www.selenium.dev/>
4. Cypress: <https://www.cypress.io/>
5. Playwright: <https://playwright.dev/>
6. TestRTC (Cyara): <https://testrtc.com/>
7. Loadero: <https://loadero.com/>
8. KITE: <https://github.com/webrtc/KITE>

WebRTC, coletando e persistindo a telemetria automaticamente através de seu módulo *Puppeteer* sem a necessidade de configurações adicionais.

- **Plataformas SaaS comerciais especializadas (ex.: TestRTC/Cyara, Loadero):** Soluções robustas que permitem simular milhares de usuários, validando a infraestrutura em larga escala. Contudo, são ferramentas proprietárias, complexas e de alto custo, voltadas para clientes *Enterprise*. O Stream Sentry foca na validação ágil e no controle de qualidade (QA) durante o ciclo de desenvolvimento, oferecendo execução viável em infraestrutura própria sem custos de licença.
- **Ferramentas Open-Source específicas (ex.: KITE):** Apesar de populares no nicho WebRTC para validação de interoperabilidade entre navegadores, costumam apresentar configuração complexa, arquitetura pesada e carecem de uma interface de visualização focada na *Developer Experience* (DX). O Stream Sentry entrega um *Dashboard* analítico moderno e intuitivo, totalmente alinhado aos fluxos de trabalho ágeis.

FORA DO ESCOPO

Os seguintes itens foram analisados e explicitamente excluídos da abrangência do Stream Sentry:

- **Testes de estresse massivo:** Simulações envolvendo milhares de usuários simultâneos focadas em sobrecarregar servidores de vídeo, uma vez que o sistema prioriza a garantia da qualidade e estabilidade de instâncias com dezenas de usuários rodando em *hardware* convencional.
- **Integração direta como plugin de CI/CD:** O desenvolvimento de executáveis nativos (*runners*) embutidos para esteiras de Integração Contínua (ex.: *GitHub Actions* nativo), embora os *outputs* do sistema (JSON/CSV) sejam projetados para ser plenamente consumíveis por essas ferramentas.
- **Testes de segurança cibernética:** Verificação de vulnerabilidades da aplicação (ex.: *pentest*, injeção de código, interceptação de mídia criptografada), pois não representam o escopo de análise de performance de rede e vídeo.

1. webrtc-internals — ferramenta interna do Chrome: <https://webrtc.org/>
2. WebRTC Statistics API (getStats): <https://www.w3.org/TR/webrtc-stats/>
3. Selenium: <https://www.selenium.dev/>
4. Cypress: <https://www.cypress.io/>
5. Playwright: <https://playwright.dev/>
6. TestRTC (Cyara): <https://testrtc.com/>
7. Loadero: <https://loadero.com/>
8. KITE: <https://github.com/webrtc/KITE>

- **Interface mobile nativa:** Desenvolvimento de aplicativos iOS/Android ou *design* ultra-responsivo focado em smartphones para o *Dashboard*, visto que a operação de orquestração técnica e visualização de *logs* ocorrerá primordialmente via navegadores em *desktops*.
- **Suporte a tecnologias legadas:** Compatibilidade com protocolos de *streaming* ou videoconferência obsoletos, como *Flash* ou implementações descontinuadas do padrão RTMP.

GESTORES, USUÁRIOS E OUTROS INTERESSADOS

Nome	Rafael Parreira Chequer
Qualificação	Estudante
Responsabilidades	Desenvolvedor do projeto, responsável por planejar, implementar e documentar o Stream Sentry.

Nome	Lucas (Persona)
Qualificação	Desenvolvedor React
Responsabilidades	Busca automatizar testes <i>end-to-end</i> para garantir estabilidade e reduzir bugs em aplicações WebRTC sem depender de ferramentas manuais demoradas.

1. webrtc-internals — ferramenta interna do Chrome: <https://webrtc.org/>
2. WebRTC Statistics API (getStats): <https://www.w3.org/TR/webrtc-stats/>
3. Selenium: <https://www.selenium.dev/>
4. Cypress: <https://www.cypress.io/>
5. Playwright: <https://playwright.dev/>
6. TestRTC (Cyara): <https://testrtc.com/>
7. Loadero: <https://loadero.com/>
8. KITE: <https://github.com/webrtc/KITE>

Nome	Ana (Persona)
Qualificação	Engenheira de QA
Responsabilidades	Responsável por validar a qualidade de áudio/vídeo, executar testes automatizados e analisar métricas de latência e falhas através de relatórios detalhados.

Nome	Felipe (Persona)
Qualificação	Tech Lead / Equipe Ágil
Responsabilidades	Acompanha a estabilidade do produto ao longo do ciclo de desenvolvimento por meio do <i>dashboard</i> e do histórico de testes, e exporta relatórios em JSON/CSV para compartilhar resultados com a equipe e integrá-los a ferramentas externas de análise.

LEVANTAMENTO DE NECESSIDADES

1. Automação de testes *end-to-end* com suporte multi-API

Justificativa: Desenvolvedores e equipes ágeis precisam reduzir o tempo e o esforço

1. webrtc-internals — ferramenta interna do Chrome: <https://webrtc.org/>
2. WebRTC Statistics API (getStats): <https://www.w3.org/TR/webrtc-stats/>
3. Selenium: <https://www.selenium.dev/>
4. Cypress: <https://www.cypress.io/>
5. Playwright: <https://playwright.dev/>
6. TestRTC (Cyara): <https://testrtc.com/>
7. Loadero: <https://loadero.com/>
8. KITE: <https://github.com/webrtc/KITE>

gastos em testes manuais de videoconferências, garantindo que o sistema valide cenários reais nos principais provedores do mercado de forma automatizada e sem configurações complexas.

2. **Monitoramento de qualidade e telemetria em tempo real**

Justificativa: O diagnóstico de instabilidades em aplicações de vídeo exige visibilidade instantânea do comportamento da rede e da mídia, permitindo a identificação de gargalos sem a necessidade de aguardar o término completo da execução do teste.

3. **Análise estruturada de dados e integração ágil**

Justificativa: Engenheiros de QA e gestores necessitam de métricas claras, consolidadas e fáceis de exportar para identificar problemas rapidamente e garantir a interoperabilidade dos dados com ferramentas de análise de terceiros ou esteiras de CI/CD.

FUNCIONALIDADES DO PRODUTO

Necessidade: Automação de testes end-to-end com suporte multi-API	
Funcionalidade	Categoria
1. Simulação de múltiplos usuários virtuais simultâneos utilizando <i>headless browsers</i> (Puppeteer) otimizados para execução em hardware padrão.	Crítico
2. Execução de testes com integração nativa e intercambiável para provedores distintos (WebRTC nativo, Whereby, Zoom SDK e Jitsi).	Crítico

1. webrtc-internals — ferramenta interna do Chrome: <https://webrtc.org/>
2. WebRTC Statistics API (getStats): <https://www.w3.org/TR/webrtc-stats/>
3. Selenium: <https://www.selenium.dev/>
4. Cypress: <https://www.cypress.io/>
5. Playwright: <https://playwright.dev/>
6. TestRTC (Cyara): <https://testrtc.com/>
7. Loadero: <https://loadero.com/>
8. KITE: <https://github.com/webrtc/KITE>

Necessidade: Monitoramento de qualidade e telemetria em tempo real	
Funcionalidade	Categoria
1. Coleta e extração contínua de métricas de baixo nível (latência, falhas de conexão, <i>bitrate</i>) diretamente das instâncias virtuais.	Crítico
2. Transmissão de telemetria assíncrona via WebSockets para atualização de gráficos e logs no <i>Dashboard</i> em tempo real.	Importante
3. Injeção de cenários de instabilidade de rede para testar o comportamento e a resiliência da aplicação de vídeo (<i>stress test</i>).	Importante

Necessidade: Análise estruturada de dados e integração ágil	
Funcionalidade	Categoria
1. Geração de relatórios analíticos com métricas detalhadas (tempo de conexão, falhas, cobertura), exportáveis em formatos estruturados como JSON ou CSV.	Crítico
2. Manutenção de um histórico visual interativo contendo todas as execuções de teste e seus respectivos status (Sucesso/Falha) para auditoria.	Importante

INTERLIGAÇÃO COM OUTROS SISTEMAS

O **Stream Sentry** é uma ferramenta autônoma (*standalone*), mas sua arquitetura foi desenhada para interagir de forma fluida com outros ecossistemas:

1. webrtc-internals — ferramenta interna do Chrome: <https://webrtc.org/>
2. WebRTC Statistics API (getStats): <https://www.w3.org/TR/webrtc-stats/>
3. Selenium: <https://www.selenium.dev/>
4. Cypress: <https://www.cypress.io/>
5. Playwright: <https://playwright.dev/>
6. TestRTC (Cyara): <https://testrtc.com/>
7. Loadero: <https://loadero.com/>
8. KITE: <https://github.com/webrtc/KITE>

-
- **Headless Browsers:** utiliza a biblioteca Puppeteer para instanciar e controlar navegadores reais (Chromium) em segundo plano, injetando fluxos de áudio e vídeo simulados nas chamadas.
 - **Provedores de Videoconferência:** por meio do padrão de projeto Strategy (ProviderFactory/IConferenceProvider), o runner identifica e interage com diferentes plataformas para ingressar nas salas e coletar a telemetria, com suporte intercambiável a Jitsi, Whereby, Zoom e salas WebRTC genéricas.
 - **Banco de Dados (MongoDB):** persiste os dados de usuários, o histórico de testes (coleção test_history) e os eventos de telemetria (coleção telemetry_events).
 - **Ferramentas de Análise e CI/CD:** gera saídas analíticas padronizadas em JSON e CSV, que podem ser consumidas por ferramentas externas de análise, observabilidade ou esteiras de automação (CI/CD).

RESTRIÇÕES

O desenvolvimento e a operação do sistema estão sujeitos às seguintes restrições técnicas e operacionais:

- **Compatibilidade do Cliente:** A interface administrativa (*Dashboard*) e os *workers* gerados pelo Puppeteer devem rodar obrigatoriamente em ambientes que possuam suporte integral às APIs modernas do HTML5, WebRTC e WebSockets.
- **Desempenho e Escalabilidade Física:** O processamento e a decodificação de múltiplos fluxos de vídeo no modo *headless* são intensivos em CPU e RAM. Portanto, a ferramenta está restrita à simulação de cenários de QA com dezenas de usuários simultâneos (ex.: 1 a 50) em *hardware* padrão (ex.: processador i5/i7, 8 GB RAM), não sendo focada em testes de estresse massivo de infraestrutura.
- **Acessibilidade de Interface:** O painel de controle (*Dashboard*) é projetado prioritariamente para acessos via *desktops* em monitores com resoluções adequadas para a análise de painéis de dados complexos, não havendo suporte prioritário para visualização em *smartphones*.

1. webrtc-internals — ferramenta interna do Chrome: <https://webrtc.org/>
2. WebRTC Statistics API (getStats): <https://www.w3.org/TR/webrtc-stats/>
3. Selenium: <https://www.selenium.dev/>
4. Cypress: <https://www.cypress.io/>
5. Playwright: <https://playwright.dev/>
6. TestRTC (Cyara): <https://testrtc.com/>
7. Loadero: <https://loadero.com/>
8. KITE: <https://github.com/webrtc/KITE>

-
- **Licenciamento e Limitações Legais:** O sistema opera sob as regras de limitação de taxa (*rate limits*) e termos de serviço das APIs de vídeo de terceiros (ex.: Zoom SDK). O usuário final é responsável por fornecer *tokens* válidos e respeitar as cotas de sua respectiva conta.

DOCUMENTAÇÃO

Para garantir a manutenibilidade, o uso adequado e a avaliação acadêmica rigorosa, os seguintes artefatos serão fornecidos:

- **Documento de Visão:** O presente documento, delimitando o escopo funcional, o perfil de usuários e os diferenciais técnicos do sistema (formato .md).
- **Documentação Técnica e de Arquitetura:** Diagramas UML, explicação da organização estrutural, detalhamento do uso do padrão *Strategy* para os provedores de vídeo e o fluxo de comunicação via WebSockets.
- **Manual do Usuário:** Guia passo a passo descrevendo como configurar o ambiente local, parametrizar simulações, executar testes e interpretar as métricas dos relatórios.
- **Código-fonte:** Repositório completo disponibilizado publicamente sob a licença MIT, permitindo auditoria e contribuição por parte da comunidade acadêmica e de desenvolvedores.

1. webrtc-internals — ferramenta interna do Chrome: <https://webrtc.org/>
2. WebRTC Statistics API (getStats): <https://www.w3.org/TR/webrtc-stats/>
3. Selenium: <https://www.selenium.dev/>
4. Cypress: <https://www.cypress.io/>
5. Playwright: <https://playwright.dev/>
6. TestRTC (Cyara): <https://testrtc.com/>
7. Loadero: <https://loadero.com/>
8. KITE: <https://github.com/webrtc/KITE>